UNIVERSIDAD CAECE

Cámara Argentina de Comercio y Servicios

LICENCIATURA EN CIENCIAS DE DATOS

ESTRUCTURAS Y ALGORITMOS DE DATOS II

GUIA DE LA UNIDAD Nro. 1

Contenidista: Maximiliano López

ÍNDICE

1- INTRODUCCIÓN 2

2- DESARROLLO 3

3- CONCLUSIONES 17

4- BIBLIOGRAFÍA DE REFERENCIA 17

FIGURAS:

Ilustración 1 Tiempo de A 6
Ilustración 2 Búsqueda secuencial ordenada 7
Ilustración 3 O(g(n)) 9
Ilustración 4 Ω(g(n)) 10
Ilustración 5 Θ(g(n)) 11
Ilustración 6 Principales funciones de orden 12
Ilustración 7 Búsqueda Secuencial Ordenada 14
Ilustración 8 Búsqueda Binaria 15
Ilustración 9 Recorrido de una matriz cuadrada 16

1- INTRODUCCIÓN

Comenzamos nuestra materia y para eso le proponemos realizar un repaso sobre los conceptos básicos vistos en la materia anterior sobre Algoritmos. Este repaso le permitirá focalizarse en las características de un algoritmo, en particular en aquella a la que nos dedicaremos en esta unidad, la eficiencia.

2- DESARROLLO

1.1 ANÁLISIS DE ALGORITMOS

Repasando la definición de algoritmos, podrá recordar que es **una lista finita de instrucciones que describe paso a paso y de manera precisa un proceso que garantiza que, al finalizar, resuelve un problema perteneciente a una familia de problemas**.

De acuerdo con esto, podrá encontrar en la misma definición las características que debe tener todo algoritmo

- **Es finito**. Todo algoritmo está compuesto por una lista finita de instrucciones. Es decir que la cantidad de acciones del algoritmo siempre está limitada, todo algoritmo tiene un Inicio y un Final.
- **Es Preciso**. La lista de acciones es bien clara y precisa, no debe haber ambigüedades ni dejar valores o instrucciones sobreentendidas o presupuestas. Para lograr esta precisión trabajaremos con un lenguaje especial para escribir nuestros algoritmos, definiendo las normas de sintaxis y semántica. Este lenguaje se llama seudocódigo, y comenzaremos a trabajar con él en esta misma unidad.
- **Finaliza**. Todo algoritmo comienza en un tiempo determinado y en algún momento finaliza su ejecución. Es decir que todo algoritmo siempre tiene que terminar, no puede haber algoritmos infinitos. Por ejemplo, pensemos en listar todos los números enteros, esto no sería un algoritmo, como ya estará pensando, porque como hay infinitos números enteros, este listado no finalizaría nunca.

- **Es efectivo**. El algoritmo resuelve un problema que forma parte de un conjunto o familia de problemas. Todo algoritmo debe funcionar correctamente, generando las salidas esperables y dando solución al problema.

Además … podemos decir que un algoritmo

- **ES EFICIENTE** cuando consume menos recursos de espacio y tiempo

La eficiencia es el tema fundamental de esta unidad. Así es como le brindaremos los recursos necesarios para que usted pueda evaluar la complejidad algorítmica de un algoritmo.

1.1.1 COMPLEJIDAD ALGORÍTMICA

La Complejidad algorítmica es la medida de la cantidad de recursos temporales que necesita un algoritmo.
Gracias a la complejidad algorítmica usted podrá:

- Comprender el tiempo y el espacio que se utiliza para ejecutar un algoritmo.
- Determinar la eficiencia de un algoritmo.

1.1.2 CRITERIOS

A continuación, le brindaremos una serie de criterios que le serán de utilidad para analizar la complejidad algorítmica.

- Lo primero que deberá tener en cuenta en estos criterios es que la complejidad no brinda una medida absoluta, sino que es **relativa al tamaño de los datos de entrada**.
- Por otro lado, el Tiempo, debe ser **independiente** de un equipo de hardware, lenguaje o cualquier otro elemento de software. Es decir que el tiempo no mide lo que “tarda” un algoritmo en un determinado equipo, poque eso puede variar

dependiendo de los recursos, velocidad y demás parámetros de configuración. Es por eso que la medida de Tiempo es una medida de lo que consume ese algoritmo independientemente de donde sea implementado.

- Para lograr esa independencia del ambiente es que se puede medir el Tiempo en **Pasos** del algoritmo.
- Otro criterio importante está dado con el costo del algoritmo, que se encuentra relacionado con el tamaño de los datos. A la hora de evaluar el costo se deben tener en cuenta los siguientes casos posibles:
    - El mejor caso
    - El peor caso
    - El caso promedio

Ahora que ya le presentamos los criterios para analizar la complejidad algorítmica, pasemos a ver con mayor detalle cómo calcularla.

1.1.3 TIEMPO DE EJECUCIÓN

Como le comentamos previamente, el Tiempo requerido por un algoritmo depende del tamaño de los datos. Por esta razón, la complejidad temporal se expresa de la siguiente manera:

$$T(n)$$

Ahora bien, es importante aclarar cómo se representa el tamaño de los datos ya que, dependiendo del algoritmo, puede ser:

- El dato en sí. Por ejemplo, si es un número entero, 1, -5, 200.
- La cantidad de elementos que lo componen. Como puede ser en el caso de un arreglo o un conjunto de elementos.

Ahora le proponemos que analice el siguiente ejemplo para entender el Tiempo en función del dato.

```
Suma <-  0
Mientras (A>0) Hacer
    Suma <-  Suma + A
     A <- A-1
FinMientras
```

Ilustración 1 Tiempo de A

En este algoritmo, la cantidad de iteraciones estará dada por el valor que tenga A. Así, si A = 10, el ciclo se repetirá 10 veces. Por otro lado, si A tiene un valor de 500, el ciclo de repetirá 500 veces. En este ejemplo queda claro que los pasos que dará el algoritmo son independientes del hardware o de cualquier otro recurso, ya que solo depende de la lógica con la que se programó, y en este caso será igual a el valor que tenga A.

En función a esto que acabamos de analizar podemos establecer que el tiempo depende de A. Es decir:

$$T(A)$$

1.1.4 ANÁLISIS DE CASOS

Ahora que ya conoce los criterios para establecer el Tiempo de un algoritmo, le presentaremos un ejemplo para explicarle cómo se realiza un análisis de los casos de un algoritmo.

Comencemos con la Búsqueda secuencial ordenada.

```
POS <-1
ESTA <- FALSO
Mientras (POS <= CE) Y (NO ESTA) Y (A[POS] <= DATO) Hacer
   Si (A[POS] = DATO) entonces
     ESTA <- VERDADERO
   sino
     POS <- POS + 1
  finsi
FinMientras
```

Ilustración 2 Búsqueda secuencial ordenada

Podemos definir los siguientes casos.

- **Mejor Caso**: **DATO** está en la primera posición.
- **Peor Caso**: **DATO** es mayor que el elemento mayor o bien **DATO** está en la última posición.
- **Caso Promedio: DATO** está o NO está en una posición equiprobable. Esta situación sería el caso general en la que el dato a buscar se encuentra en cualquier posición, exceptuando la primera o la última porque corresponden a los otros casos.

Es fundamental establecer los tres casos, ya que a partir de este análisis usted podrá establecer el comportamiento para las distintas situaciones que se podrán presentar.

1.2 FUNCIONES DE COMPLEJIDAD

Seguramente ya habrá notado al realizar el análisis de casos que habrá una diferencia de comportamiento entre los diversos casos. La cuestión es definir qué

medida se utilizará para definir el comportamiento del algoritmo.

Así es como se define el orden de la función T (n), que expresa el comportamiento dominante para los datos de gran tamaño.

Para poder determinar y expresar este comportamiento, denominado **Comportamiento Asintótico,** se utilizan las notaciones que le presentamos a continuación.

1.2.1 NOTACIÓN DE ORDEN

Notación para expresar el Orden de la Función T(n), que es el comportamiento dominante para los datos de gran tamaño.

- O mayúscula
- Ω Omega
- Θ Theta

1.2.2 O MAYÚSCULA (COTA SUPERIOR)

El conjunto ***O(g(n))*** está formado por aquellas funciones ***f(n)*** que crecen a un ritmo menor o igual que el de ***g(n).***

Matemáticamente se puede expresar con la siguiente fórmula

$$O(g(n)) = \{f:N \rightarrow R+ \mid \exists\, c \text{ constante positiva y } n0 \in N : f(n) \leq cg(n),\ \forall n \geq n0\}$$

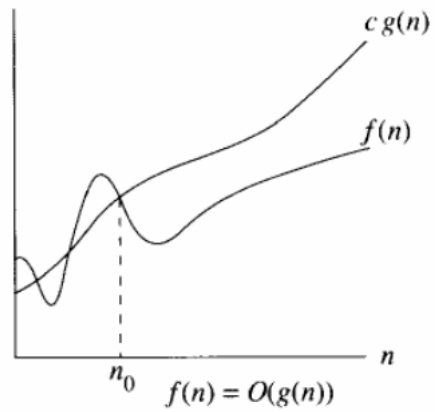

Ilustración 3 O(g(n))

Esto significa que ***f*** no crece más rápido que ***g***. Así es como usted puede ver que se denomina Cota Superior porque se limita superiormente el comportamiento asintótico de la función.

1.2.3 OMEGA Ω (COTA INFERIOR)

El conjunto ***Ω (g(n))*** está formado por aquellas funciones ***f(n)*** que crecen a un ritmo mayor o igual que el de ***g(n)***.

Matemáticamente se puede expresar con la siguiente fórmula.

$$\Omega(g(n)) = \{f:N \rightarrow R+ \mid \exists\, c \text{ constante positiva y } n0 \in N : 0 < c\,g(n) \leq f(n),\ \forall n \geq n0\}$$

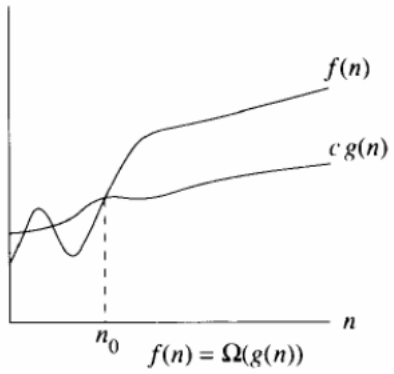

Ilustración 4 Ω(g(n))

Esto significa que ***f*** crece más rápido que ***g***. Así es como usted puede ver que se denomina Cota Inferior porque se limita inferiormente el comportamiento asintótico de la función.

1.2.4 THETA Θ (COTA AJUSTADA)

El conjunto ***Θ(g(n))*** está formado por aquellas funciones ***f(n)*** que crecen exactamente al mismo ritmo que ***g(n).***
Matemáticamente se puede expresar con la siguiente fórmula.

$$\Theta(g(n)) = \{f:N \rightarrow R+ \mid \exists\ c \text{ constante positiva y } n0 \in N : 0 < c1g(n) \leq f(n) \leq c2g(n),\ \forall\ n \geq n0\}$$

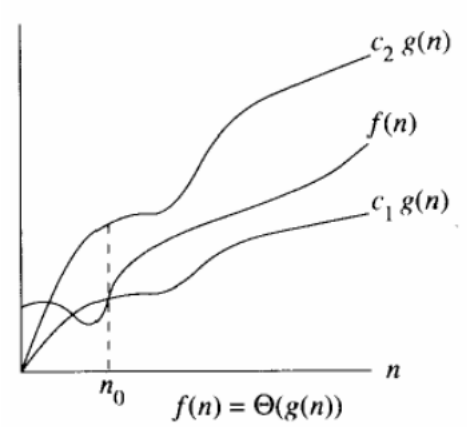

Ilustración 5 Θ(g(n))

Esto significa que ***f*** crece al mismo ritmo que ***g***. Así es como usted puede ver que se denomina Cota Ajustada porque mantiene el mismo crecimiento del comportamiento asintótico de la función.

1.3 ORDEN DE COMPLEJIDAD

Ahora que ya conoce las distintas funciones de complejidad, le explicaremos qué es el Orden de Complejidad. Para eso es importante comprender que la familia O(f(n)), Ω(f(n)), Θ(f(n)) define un orden de complejidad representado por la función f(n) más sencilla.

A continuación, le presentamos las principales funciones de orden.

Orden	Nombre
O(1)	constante
O(log n)	logarítmica
O(n)	lineal
O(n log n)	casi lineal
O(n^2)	cuadrática
O(n^3)	cúbica
O(a^n)	exponencial

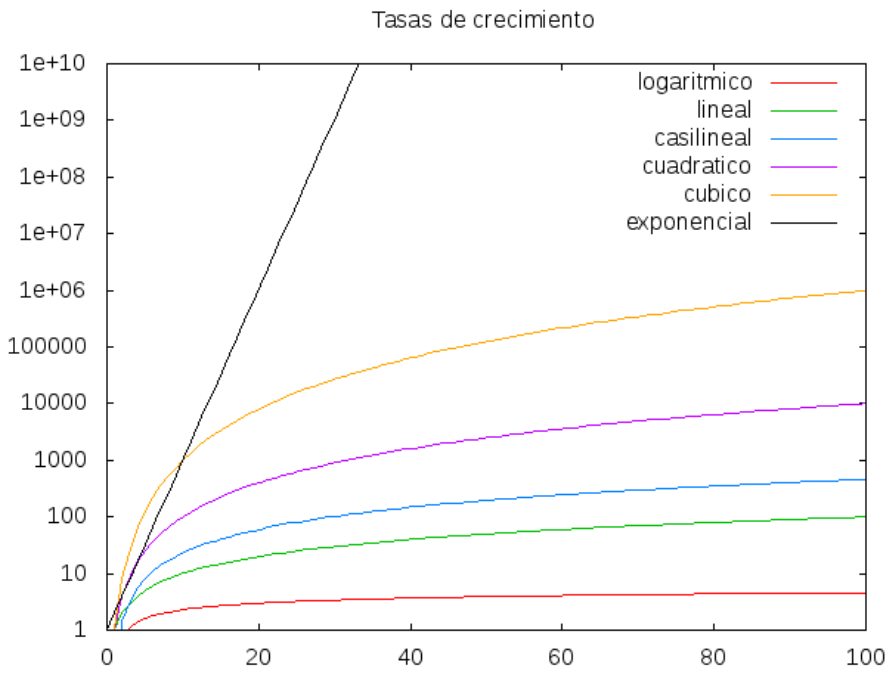

Ilustración 6 Principales funciones de orden

1.3.1 REGLAS DE CÁLCULO

Ahora que llegó hasta este momento, estamos en condiciones de pasar a la práctica y presentarle algunas reglas para poder calcular el orden de complejidad.

- Las instrucciones simples son de **O(1).**
  Ejemplos de estas instrucciones son Asignaciones, comparaciones, lectura, escritura de datos simples. Acceso a componentes de un arreglo o de un registro.

- El tiempo de una secuencia de instrucciones se obtiene sumando los tiempos de cada instrucción.
  T(S1;S2) = T(S1) + T(S2)

  O(T(S1 ; S2)) = max(O(T(S1), T(S2)))

- Condicionales. El tiempo de ejecución de una instrucción "if then else" es:
  O(T(IF-THEN-ELSE)) = max(O(T(condición)), max(O(T(rama THEN)), O(T(rama ELSE))))

- Según. El tiempo de ejecución de una instrucción "switch - case" es:
  O(T(SWITCH)) = max(O(T(condición), T(S1), …T(Sn)))

- Ciclos. El tiempo de ejecución de una instrucción "while – do while - for" está definido por la cantidad de iteraciones. Es equivalente para todas las estructuras iterativas y se puede calcular así
  O(T(BUCLE)) = (#iteraciones) * max(O(T(condición), T(cuerpo))

- Rutinas no recursivas. En el caso de las rutinas se aplican las reglas anteriores. Teniendo en cuenta entonces que el orden se corresponderá con el orden de la secuencia de las instrucciones de la rutina, es decir.
  O(T(RUTINA)) = max(O(T(S1), …T(Sn)))

1.4 EJEMPLOS

Ha llegado el momento de poner todos los conocimientos en práctica, y para eso le presentaremos una serie de ejemplos en donde le explicaremos cómo calcular el Orden de Complejidad.

1.4.1 BÚSQUEDA SECUENCIAL ORDENADA

```
POS <-1
ESTA <- FALSO
Mientras (POS <= CE) Y (NO ESTA) Y (A[POS] <= DATO) Hacer
   Si (A[POS] = DATO) entonces
     ESTA <- VERDADERO
   sino
     POS <- POS + 1
  finsi
FinMientras
```

Ilustración 7 Búsqueda Secuencial Ordenada

Como vimos previamente, el Peor caso es el que el dato no se encuentra en el arreglo y es mayor al último elemento, o bien el dato es el último elemento. En ambos casos, será dependiente del orden del tamaño del arreglo por lo tanto podemos establecer que la complejidad algorítmica es

$$O(n)$$

1.4.2 BÚSQUEDA BINARIA

```
IZQ <-1
DER <- CE
ESTA <- FALSO
Mientras (IZQ <= DER) Y (NO ESTA) Hacer
    MEDIO = (IZQ + DER) DIV 2
    Si (A[MEDIO] < DATO) entonces
        IZQ <- MEDIO + 1
    Sino
        Si (A[MEDIO] > DATO) entonces
            DER <- MEDIO -1
        Sino
            ESTA <- VERDADERO
            POS <- MEDIO
        Finsi
    finsi
FinMientras
```

Ilustración 8 Búsqueda Binaria

En este algoritmo se va dividendo al arreglo a la mitad en cada paso, para descartar la mitad del arreglo en donde ya no se podrá encontrar el dato. Eso significa que, si bien la complejidad crece a medida que aumenta el tamaño del arreglo, no lo hace en el mismo orden de N, sino en un orden menor. Ese orden menor se estable con la función log(n). Así es como, en el peor de los casos, la complejidad algorítmica es

$$O(\log n)$$

1.4.3 MOSTRAR MATRIZ CUADRADA

Tomemos el siguiente ejemplo en el que se recorre toda una matriz cuadrada y se muestra cada uno de sus datos.

```
Para C <- 1 hasta CANT Hacer
    Para F <- 1 hasta CANT Hacer
        Mostrar M[F,C]
    FinPara
FinPara
```

Ilustración 9 Recorrido de una matriz cuadrada

Como habrá visto, la cantidad de pasos que realizan estos dos ciclos anidados depende del valor de CANT, es decir de la cantidad de filas y columnas de la matriz. Y al estar en un ciclo anidado, el crecimiento de los pasos es mucho mayor a medida que va creciendo la dimensión de la matriz. Por ejemplo, en una matriz de 2x2, se recorren 4 elementos; mientras que en una matriz de 3x3 se recorren 9. Así es como este crecimiento se puede asociar al cuadrado de las dimensiones, por lo tanto, la complejidad algorítmica es

$$O(n^2)$$

3- CONCLUSIONES

En esta unidad le explicamos que todo ALGORITMO es EFICIENTE cuando utiliza la menor cantidad de ESPACIO y TIEMPO. Usted podrá medir la eficiencia de un algoritmo por medio de la COMPLEJIDAD ALGORÍTMICA, estableciendo en el ANÁLISIS DE CASOS, el MEJOR, PEOR y el caso PROMEDIO. Esa complejidad se establece de acuerdo con una notación establecida en el ORDEN DE COMPLEJIDAD.

4- BIBLIOGRAFÍA DE REFERENCIA

Bisbal Riera, J. (2013). Manual de algorítmica: recursividad, complejidad y diseño de algoritmos. Editorial UOC.
https://elibro.net/es/ereader/caece/56561?page=50

Sedgewick, R. y Wayne, K. (2011). Algorithms. 4th Edition. Addison- Wesley.

Cormen, T., Leiserson, C., Rivest, R. & Stein, C. (2022) Introduction to algorithms. 4th Edition. The MIT Press.